LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1

ANDROID BASIC WITH COMPOSE

Oleh: Anggraeni Dwi Zahra
NIM. 2410817220018

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basic with Kotlin ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : AnggraeniDwi Zahra
NIM : 2410817220018

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 19930703 201903 01 011

DAFTAR ISI

DAFTAR ISI 3

DAFTAR GAMBAR 4

DAFTAR TABEL 5

SOAL 1 6

A. Source Code 9

B. Output Program 15

C. Pembahasan 18

D. Tautan Git 22

Additional: 23

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi 6
Gambar 2. Tampilan Dadu Setelah Di-Roll 7
Gambar 3. Tampilan Roll Dadu Double 8
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose (Dadu Kosong) 15
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose (Dadu Tidak Kembar) 15
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose (Dadu Double) 16
Gambar 7. Screenshot Hasil Jawaban Soal 1 XML (dadu kosong) 16
Gambar 8. Screenshot Hasil Jawaban Soal 1 XML (Dadu Tidak Kembar) 17
Gambar 9. Screenshot Hasil Jawaban Soal 1 XML (dadu double) 17

DAFTAR TABEL

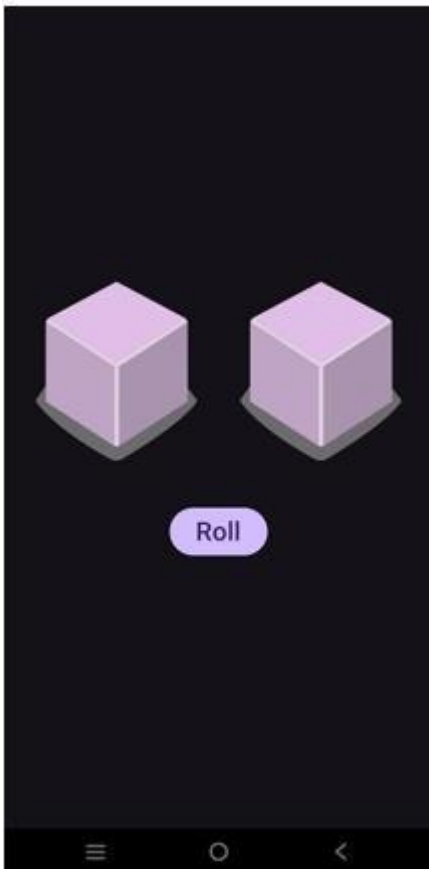
Tabel 1. Source Code Mainactivity XML Soal 1 9
Tabel 2. Source Code Mainactivity Compose Soal 1 10
Tabel 3. Source Code HomeScreen Compose Soal 1 11
Tabel 4. Source Code activity_main XML Soal 1 14

SOAL 1

Soal Praktikum:

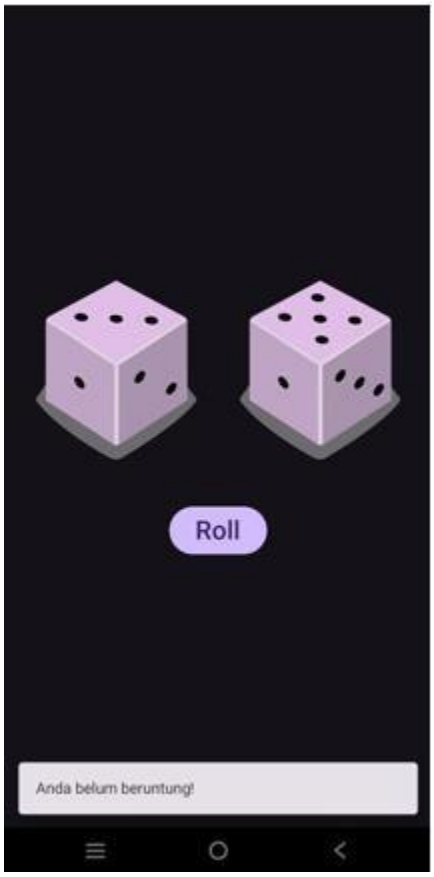
Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah- ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



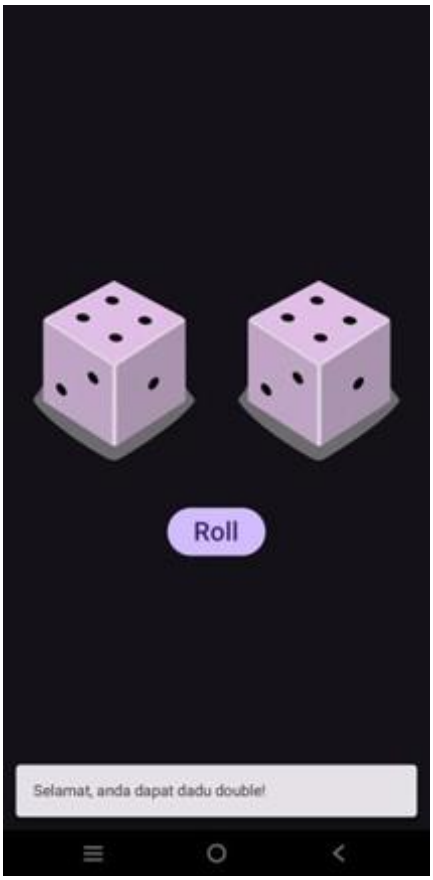
Gambar 1. Tampilan Awal Aplikasi

2. Setelah user menekan tombol “Roll” maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2 maka aplikasi akan menampilkan pesan “Anda belum beruntung!” seperti dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah Di-Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat, anda dapat dadu double!” seperti yang dapat dilihat pada Gambar 3.



Gambar 3. Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder Modul 1** dalam bentuk Project. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut: https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp=sharing%20%20

A. Source Code

MainActivity.kt

MainActivity.kt XML

Tabel 1. Source Code Mainactivity XML Soal 1

```
package com.example.diceroller

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import android.widget.Button
import android.widget.ImageView
import android.widget.Toast
import com.example.diceroller.ui.theme.DiceRollerTheme

// Aktifkan setContentView di bawah ini untuk menjalankan versi XML
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContentView(R.layout.activity_main)

        val rollButton: Button = findViewById(R.id.roll_button)
        val diceImage1: ImageView = findViewById(R.id.dice_image_1)
        val diceImage2: ImageView = findViewById(R.id.dice_image_2)

        rollButton.setOnClickListener {
            rollDice(diceImage1, diceImage2)
        }
    }
```

```

    private fun rollDice(img1: ImageView, img2:
ImageView) {
        val result1 = (1..6).random()
        val result2 = (1..6).random()

        img1.setImageResource(getDiceResource(result1))
        img2.setImageResource(getDiceResource(result2))

        val message = if (result1 == result2) {"Selamat,
anda dapat dadu double!"}
        else {"Anda belum beruntung!"}

        Toast.makeText(this, message,
Toast.LENGTH_SHORT).show()    }

    private fun getDiceResource(number: Int): Int {
        return when (number) {
             1 -> R.drawable.dice_1
             2 -> R.drawable.dice_2
             3 -> R.drawable.dice_3
             4 -> R.drawable.dice_4
             5 -> R.drawable.dice_5
             6 -> R.drawable.dice_6
             else -> R.drawable.dice_0
        }
    }
}
```

MainActivity.kt Compose

Tabel 2. Source Code Mainactivity Compose Soal 1

```
package com.example.diceroller

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
```

```
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import android.widget.Button
import android.widget.ImageView
import android.widget.Toast
import com.example.diceroller.ui.theme.DiceRollerTheme

// Hapus komentar setContent di bawah ini untuk menjalankan versi Jetpack Compose

/* class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            DiceRollerTheme {
            HomeScreen() // Panggil fungsi dari file HomeScreen.kt
            }
        }
    }
} */
```

HomeScreen.kt Compose

Tabel 3. Source Code HomeScreen Compose Soal 1

```
package com.example.diceroller

import android.widget.Toast
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.Scaffold
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
```

```
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.diceroller.ui.theme.DiceRollerTheme
import androidx.compose.foundation.layout.Row

@Composable
fun HomeScreen(modifier: Modifier = Modifier) {
    val context = LocalContext.current
    var result1: Int by remember { mutableStateOf(0) }
    var result2: Int by remember { mutableStateOf(0) }

    fun getDiceResource(result: Int): Int {
        return when(result) {
            1 -> R.drawable.dice_1
            2 -> R.drawable.dice_2
            3 -> R.drawable.dice_3
            4 -> R.drawable.dice_4
            5 -> R.drawable.dice_5
            6 -> R.drawable.dice_6
            else -> R.drawable.dice_0
        }
    }

    val onClickRoll: () -> Unit = {
        result1 = (1..6).random()
        result2 = (1..6).random()

        val message = if (result1 == result2) {"Selamat, anda dapat dadu double!"}
        else {"Anda belum beruntung!"}

        Toast.makeText(context, message, Toast.LENGTH_SHORT).show()
    }

    Scaffold { innerPadding ->
        Column(
            modifier = Modifier
```

```
                .padding(innerPadding)
                .fillMaxSize(),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Image(
                    painter = painterResource(id = getDiceResource(result1)),
                    contentDescription = "Dadu 1",
                    modifier = Modifier.size(150.dp)
                )
                Spacer(modifier = Modifier.size(16.dp))
                Image(
                    painter = painterResource(id = getDiceResource(result2)),
                    contentDescription = "Dadu 2",
                    modifier = Modifier.size(150.dp)
                )
            }
            Spacer(modifier = Modifier.size(30.dp))
            Button(onClick = onClickRoll) {
                Text(text = "Roll")

            }
        }
    }
}

@Preview
@Composable
private fun HomeScreenPreview() {
    DiceRollerTheme {
        HomeScreen()
    }
}
```

activity_main.xml

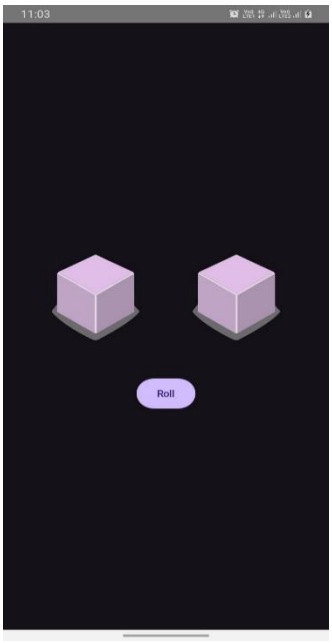
Tabel 4. Source Code activity_main XML Soal 1

```
<?xml version="1.0" encoding="UTF-8" ?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/androi
d"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#312F2F"
    android:gravity="center"
    android:orientation="vertical">

    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:orientation="horizontal">

        <ImageView
            android:id="@+id/dice_image_1"
            android:layout_width="150dp"
            android:layout_height="150dp"
            android:src="@drawable/dice_0" />

        <ImageView
            android:id="@+id/dice_image_2"
            android:layout_width="150dp"
            android:layout_height="150dp"
            android:layout_marginStart="16dp"
            android:src="@drawable/dice_0" />
    </LinearLayout>

    <View
        android:layout_width="0dp"
        android:layout_height="32dp" />

    <Button
        android:id="@+id/roll_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:backgroundTint="@color/purple_200"
        android:text="Roll"
```

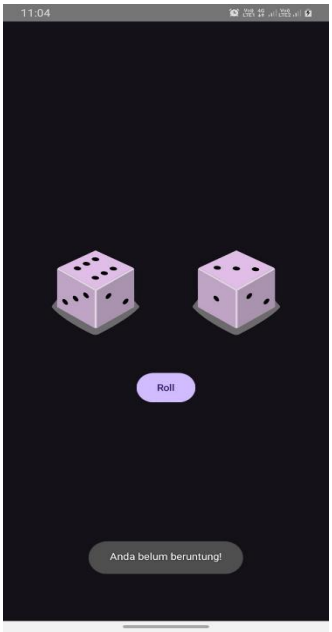
```
        android:textAllCaps="false" />

</LinearLayout>
```

B. Output Program

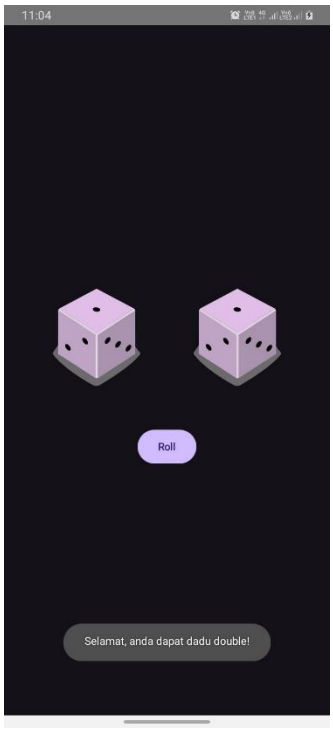
A. Compose



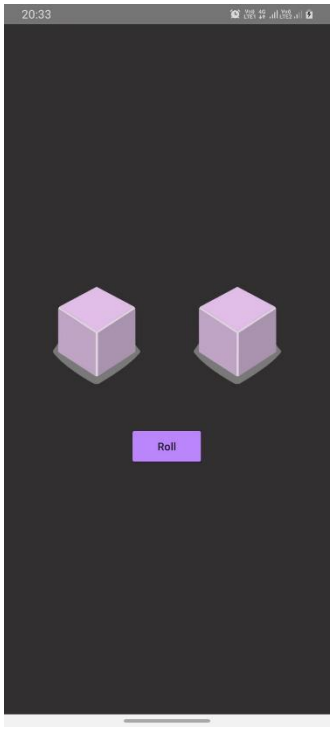
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose (Dadu Kosong)



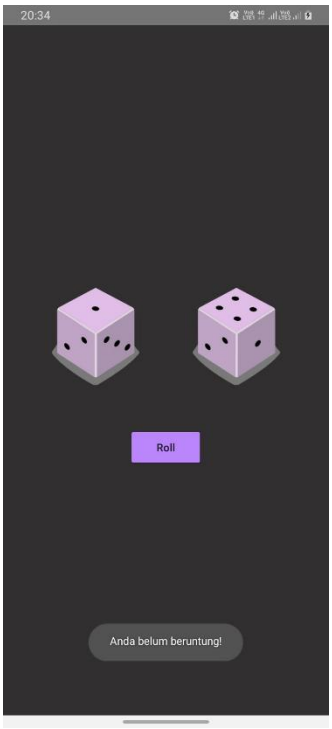
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose (Dadu Tidak Kembar)



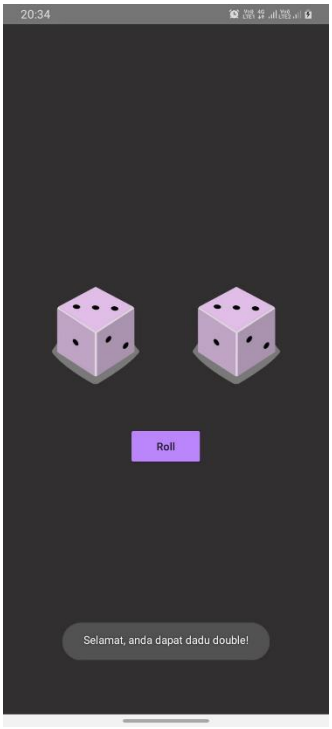
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose (Dadu Double)



Gambar 7. Screenshot Hasil Jawaban Soal 1 XML (dadu kosong)



Gambar 8. Screenshot Hasil Jawaban Soal 1 XML (Dadu Tidak Kembar)



Gambar 9. Screenshot Hasil Jawaban Soal 1 XML (dadu double)

C. Pembahasan

MainActivity.kt:

Dalam Kotlin, penulisan *package name* dan *import library* digunakan untuk mengorganisasi kode, meminimalkan konflik nama kelas, dan mempermudah pemeliharaan. Pada penerapannya, penulisan kelas diawali dengan keyword `class` kemudian dilanjutkan dengan nama kelasnyanya. Kemudian, tanda titik dua `(:)` digunakan untuk melakukan *extends* dan *implements* dari kelas tersebut maka. Pada kode ini dilakukan *extends* kelas ke `ComponentActivity`.

Pada Kotlin, penulisan nama variable harus diawali dengan nama variable-nya lalu dilanjutkan dengan tipe datanya ataupun langsung diinisialisasikan. Ada 2 tipe penulisan variable di Kotlin yakni, menggunakan keyword `val` dan `var`. Namun, dalam pembuatan aplikasi dadu kali ini, hanya menerapkan variable val , tujuannya untuk nilai konstan atau read only. Kemudian, terdapat pula penggunaan operator `?` yang berfungsi untuk mengeksekusi sebuah variable apabila nilainya non-null saja, sehingga dapat mencegah NPE atau Null Pointer Exception.

Untuk penulisan method atau function harus menggunakan keyword fun kemudian dilanjutkan dengan nama method atau function nya. Dalam hal ini digunakan Method `onCreate` sebagai titik masuk utama aplikasi untuk menyiapkan tata letak, baik melalui `setContentView` untuk versi XML maupun `setContent` untuk versi Compose.yang digunakan untuk mengambil akses ke gambar dadu yang telah di import.

activity_main.xml:

activity_main.xml adalah file XML yang digunakan untuk tata letak aktivitas utama.

Baris `<?xml version="1.0" encoding="UTF-8"?>` yang berada di paling atas file merupakan *XML Declaration* (Deklarasi XML) yang berfungsi untuk memberi tahu sistem atau perangkat lunak tentang bagaimana cara membaca file tersebut.

`LinearLayout` adalah sebuah kontainer atau "wadah" yang berfungsi untuk menyusun komponen UI di dalamnya secara berurutan dalam satu garis lurus, baik itu ke arah samping (horizontal) maupun ke bawah (vertikal).

Baris `xmlns:android="http://schemas.android.com/apk/res/android"` disebut sebagai Deklarasi *Namespace Android*. Ini adalah baris wajib di setiap file layout XML agar Android Studio memahami perintah yang kita berikan. Berikut adalah makna komponennya:

- `xmlns` (XML Namespace): Berfungsi untuk mendefinisikan sebuah ruang nama agar atribut tidak tertukar dengan sistem lain.
- `:android`: Adalah prefix atau nama panggilan. Dengan baris ini, kita bisa menulis android:id atau android:text.
- `"http://schemas.android.com/apk/res/android"`: Ini adalah URI (*Uniform Resource Identifier*) yang berfungsi sebagai alamat unik kamus atribut resmi dari Google. URI tidak terhububg ke website, melainkan hanya ID unik agar komputer tahu bahwa atribut tersebut milik Android.

Tanpa baris ini, atribut seperti `android:layout_width` tidak akan dikenali dan aplikasi akan error saat dijalankan karena sistem tidak tahu di mana harus mencari definisi dari atribut tersebut.

LinearLayout berisikan elemen-elemen sebagai berikut:

1. ImageView: Berfungsi untuk menampilkan gambar di aplikasi. Elemen ini memiliki atribut penting, yaitu `android:src` untuk memilih file gambar dari

folder res/drawable.

2. *Button*: Berfungsi sebagai elemen interaktif yang bisa diklik oleh pengguna untuk memicu suatu aksi. Dalam kode ini penggunaannya adalah pada Tombol `"Roll"` yang ditulis dengan atribut `android:text` agar tulisan berada di dalam tombol.
3. View (Komponen Dasar): Ini adalah komponen yang berfungsi sebagai *Spacer*(Pemisah) untuk memberi jarak kosong setinggi 32dp antara baris dadu dan tombol.

HomeScreen.kt Compose:

File HomeScreen.kt menggunakan paradigma Jetpack Compose, yaitu toolkit modern dari Android untuk membangun UI secara deklaratif. Berbeda dengan XML yang memisahkan desain dan logika, pada file ini desain UI dan logika bisnis ditulis langsung menggunakan kode Kotlin.

1. `@Composable Function`: Penulisan fungsi pada file ini diawali dengan annotation `@Composable`. Hal ini menandakan bahwa fungsi `HomeScreen` dapat diubah menjadi elemen UI oleh sistem Android. Fungsi ini bersifat modular, sehingga mempermudah dalam proses reusable code.
2. *State Management* (`remember` & `mutableStateOf`): Dalam Compose, UI bersifat reaktif terhadap perubahan data. Digunakan keyword `remember` dan `mutableStateOf` pada variabel `result1` dan `result2`.
   - mutableStateOf: Berfungsi sebagai state yang memantau perubahan nilai dadu. Ketika nilai ini berubah, Compose akan melakukan re-composition (menggambar ulang) bagian UI yang terkait saja.
   - remember: Berfungsi untuk menyimpan nilai state tersebut agar tidak hilang atau ter-reset saat fungsi HomeScreen dipanggil kembali oleh sistem.
3. *Layouting* (`Column, Row, & Spacer`): Karena tidak menggunakan `LinearLayout` seperti pada XML, Compose menggunakan komponen tata

letak berikut:

- `Column`: Berfungsi untuk menyusun elemen secara vertikal. Di dalamnya terdapat parameter `horizontalAlignment` dan `verticalArrangement` untuk mengatur posisi konten agar tepat berada di tengah layar.
- `Row`: Berfungsi untuk menyusun dua *Image* (dadu) secara horizontal.
- `Spacer`: Digunakan untuk memberikan jarak atau ruang kosong antar elemen dengan satuan ukuran dp.

4. `Modifier`: Setiap komponen UI dalam Compose memiliki parameter Modifier yang digunakan untuk mengatur perilaku dan dekorasi elemen, seperti `fillMaxSize()` agar tampilan memenuhi layar, `padding()` untuk memberikan jarak dalam, serta `size()` untuk menentukan dimensi gambar.
5. *Image & Button*
   - `Image`: Digunakan untuk menampilkan gambar dadu. Pengambilan aset gambar dilakukan melalui fungsi `painterResource` yang merujuk pada ID gambar dari hasil fungsi `getDiceResource`.
   - `Button`: Berbeda dengan Button pada XML, Button di Compose memiliki parameter `onClick`. Di sini, logika pengacakan angka dadu dimasukkan ke dalam lambda `onClickRoll`, sehingga saat tombol diklik, nilai `result1` dan `result2` akan diperbarui secara otomatis.
6. `@Preview`: Terdapat fungsi `HomeScreenPreview` yang dianotasi dengan `@Preview`. Fungsi ini sangat berguna bagi pengembang untuk melihat hasil desain UI secara langsung di dalam Android Studio tanpa harus menjalankan aplikasi ke emulator atau perangkat fisik (HP).
7. `Toast`: `Toast` adalah komponen UI Android yang digunakan untuk menampilkan pesan singkat kepada pengguna sebagai bentuk umpan balik (feedback). Dalam kode ini, Toast dipanggil di dalam fungsi `onClickRoll`

menggunakan perintah `Toast.makeText`.

- `Context`: Digunakan `LocalContext.current` agar sistem tahu di Activity mana notifikasi ini harus muncul.
- `Logic`: Terdapat logika pengkondisian (if-else) di mana pesan yang muncul akan dinamis. Jika angka dadu 1 dan dadu 2 sama, maka akan muncul pesan khusus, sementara jika tidak sama, akan muncul pesan lainnya.
- `Duration`: Menggunakan `Toast.LENGTH_SHORT` agar pesan hanya muncul selama beberapa detik saja sebelum menghilang secara otomatis.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat:
https://github.com/AnggraeniDwiZahra/Pemrograman-Mobile

Additional:

ESSAY

1. Riset dan *cite* **artikel/jurnal/buku** mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. **

   Dalam pemrograman, terdapat dua paradigma utama yaitu *imperative programming* dan *declarative programming* yang memiliki pendekatan berbeda dalam menyelesaikan suatu masalah, termasuk dalam pengembangan UI. *Imperative programming* adalah paradigma yang didasarkan pada langkah-langkah atau rangkaian pernyataan yang mengubah keadaan program secara bertahap hingga mencapai hasil yang diinginkan. Proses eksekusi berfokus pada urutan perintah yang dijalankan secara berurutan. Paradigma ini memiliki beberapa subdomain seperti pemrograman modular, prosedural, berorientasi objek (OOP), dan terstruktur, dengan contoh bahasa seperti Java, C++, Python, C, Pascal, dan lainnya [1].

   Declarative programming adalah pendekatan di mana program diposisikan sebagai sebuah teori dan proses komputasi merupakan hasil deduksi dari teori tersebut [2]. Berbeda dengan pendekatan imperatif yang menekankan langkah-langkah, paradigma deklaratif lebih berfokus pada hasil akhir yang ingin dicapai tanpa menjelaskan detail prosesnya.

   Dalam pengembangan UI modern, pendekatan deklaratif digunakan untuk mendeskripsikan tampilan akhir tanpa harus menjelaskan langkah-langkah detail pembuatannya. Pendekatan ini mulai banyak digunakan dalam framework modern seperti Jetpack Compose [3]. UI dapat dituliskan menggunakan kode Kotlin seperti ekspresi kondisi atau perulangan untuk mendeskripsikan hasil akhir, serta dapat memperbarui dirinya secara otomatis berdasarkan perubahan state tanpa instruksi manual seperti setText() atau setVisibility(), sehingga kode menjadi lebih ringkas dan mudah dibaca [3].

   Berdasarkan sumber yang ada, paradigma declarative programming dinilai

lebih relevan dengan perkembangan teknologi saat ini. Pendekatan ini memudahkan programmer untuk fokus pada hasil akhir tanpa harus memikirkan detail proses di balik layar. Meskipun pendekatan imperatif lebih unggul dalam kontrol performa, namun dari sisi efisiensi waktu, keterbacaan, dan kemudahan pengembangan, paradigma deklaratif lebih menguntungkan. Selain itu, penggunaan paradigma ini dalam berbagai teknologi modern semakin memperkuat bahwa pendekatan deklaratif lebih cocok untuk pengembangan perangkat lunak di masa depan.

2. Apa yang membedakan OOP pada kotlin dengan OOP pada java? *

   Pemrograman Berorientasi Objek (PBO) pada Java dan Kotlin pada dasarnya memiliki akar yang sama, yaitu mengorganisir kode ke dalam class dan object untuk membungkus data serta perilaku. Dalam Java, hampir semua elemen diperlakukan sebagai objek kecuali tipe data primitif, di mana class bertindak sebagai cetak biru kaku yang memisahkan variabel sebagai penyimpan data dan method sebagai instruksi operasional. Pendekatan ini memastikan struktur program yang teratur, namun sering kali menimbulkan duplikasi kode dan penulisan yang panjang (boilerplate) untuk fungsi-fungsi dasar [1].

   Kotlin melakukan modernisasi terhadap konsep PBO Java dengan memperkenalkan prinsip final by default. Jika pada Java sebuah class bisa diwariskan secara bebas kecuali ditandai final, Kotlin justru mengunci semua class dan mewajibkan penggunaan kata kunci open jika pengembang ingin melakukan pewarisan. Hal ini dirancang untuk menciptakan arsitektur yang lebih aman dan mencegah kesalahan desain pada hierarki subclass yang terlalu kompleks [3].

   Perbedaan mencolok lainnya terletak pada cara kedua bahasa ini mengelola data di dalam objek. Java menggunakan fields manual yang harus dilengkapi dengan getter dan setter secara terpisah, sedangkan Kotlin menyatukannya dalam konsep Property menggunakan val atau var. Selain itu, Kotlin jauh lebih aman karena memiliki sistem Null Safety untuk mencegah error umum akibat

data kosong, serta fitur internal modifier yang memberikan kontrol akses lebih baik dalam satu modul dibandingkan package-private milik Java [4].

3. Sebutkan, dan jelaskan mengenai SOLID Principles *

   SOLID merupakan lima prinsip dasar dalam pengembangan perangkat lunak berorientasi objek yang bertujuan untuk menghasilkan arsitektur yang baik, efisien, mudah dikembangkan, serta mudah dipelihara dalam jangka panjang. SOLID terdiri dari:

   1. Single Responsibility Principle menyatakan bahwa satu class hanya boleh memiliki satu tanggung jawab [5].

   2. Open Closed Principle menyatakan bahwa class harus terbuka untuk pengembangan namun tertutup untuk modifikasi [5].

   3. Liskov Substitution Principle menyatakan bahwa subclass harus dapat menggantikan superclass tanpa mengubah perilaku program [5].

   4. Interface Segregation Principle menyatakan bahwa client tidak boleh dipaksa menggunakan interface yang tidak diperlukan [5].

   5. Dependency Inversion Principle menyatakan bahwa modul tingkat tinggi tidak boleh bergantung pada modul tingkat rendah, melainkan pada abstraksi [5].

   Dengan menerapkan prinsip-prinsip ini, pengembang dapat membangun sistem yang tidak rapuh (non-fragile), reusable, dan fleksibel. Prinsip SOLID juga hadir sebagai solusi untuk mengatasi permasalahan utama dalam pengembangan perangkat lunak, yaitu Reusability, Extensibility, Sustainability, dan Maintainability (RESM).

   Secara umum, setiap prinsip dalam SOLID memiliki peran masing-masing dalam menjaga kualitas kode. SRP menekankan bahwa satu class hanya memiliki satu tanggung jawab, OCP mendorong agar class dapat dikembangkan tanpa harus diubah, dan LSP memastikan bahwa subclass dapat menggantikan superclass tanpa mengganggu program. ISP mengharuskan interface dibuat spesifik agar tidak membebani client, sedangkan DIP

menekankan pentingnya ketergantungan pada abstraksi, bukan pada implementasi konkret. Dengan menerapkan kelima prinsip ini, pengembangan perangkat lunak menjadi lebih terstruktur, mudah dipahami, serta lebih siap menghadapi perubahan di masa depan. Kotlin juga menyediakan fitur seperti delegation dan sealed class yang membantu penerapan prinsip SOLID secara lebih efektif [3].

4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

   Dalam pemrograman Kotlin, **class** standar berfungsi sebagai cetak biru (blueprint) utama untuk membangun sebuah objek. Analologinya seperti denah arsitek; denah tersebut bukanlah rumah aslinya, melainkan petunjuk detail tentang bagaimana rumah tersebut harus dibangun, termasuk desain ruangan dan fungsinya masing-masing. Class biasa ini bersifat serbaguna karena tidak hanya terbatas untuk menyimpan data, tetapi juga bisa menyertakan logika bisnis yang kompleks, perilaku khusus, hingga implementasi konstruktor sekunder. Namun, kekurangannya adalah pengembang harus menuliskan kode standar (boilerplate) seperti toString(), equals(), atau hashCode() secara manual jika ingin mengelola data di dalamnya dengan benar [6].

   Di sisi lain, Kotlin memperkenalkan konsep **Data Class** yang dirancang khusus sebagai wadah data (data container) yang jauh lebih ringkas. Berbeda dengan class biasa, Data Class tidak fokus pada logika atau perilaku, melainkan pada efisiensi penyimpanan informasi. Hanya dengan menambahkan satu kata kunci data, compiler secara otomatis akan menghasilkan fungsi penting seperti copy(), equals(), dan toString() [3]. Hal ini membuat proses koding jadi lebih cepat dan rapi. Menurut Musicant (2026), keunggulan Kotlin memang terletak pada sintaksnya yang lebih ringkas dibandingkan bahasa pemrograman lain seperti Java [7].

DAFTAR PUSTAKA

[1] U. Indahyanti and I. R. I. Astutik, *Buku Ajar Pemrograman Berorientasi Objek (PBO)*. Sidoarjo: UMSIDA Press, 2025.

[2] M. Alpuente, R. Barbuti, and I. Ramos, "Declarative Programming," Italy, 1994. [Online]. Available: https://www.programmazionelogica.it/wp-content/uploads/2015/12/GP1994-I-000-031.pdf

[3] S. Isakova, R. Elizarov, S. Aigner, and D. Jemerov, *Kotlin in Action , Second Edition MEAP V09*, 2nd ed. Manning Publications, 2024.

[4] J. J. Sanjaya and J. Susilo, "Perbandingan Performa Kotlin vs Java dalam Pengembangan Android dengan Metode Iterasi While," *bit-Tech*, vol. 7, no. 2, pp. 545–553, 2024, doi: 10.32877/bt.v7i2.1898.

[5] V. K. Madasu, T. Eltaib, and T. V. S. N. Venna, "SOLID Principles in Software Architecture and Intrduction to RESM Concept in OOP," *J. Multidiscip. Eng. Sci. Technol.*, vol. 2, no. 2, p. 3, 2015.

[6] K. Kanjiya, "Kotlin Class vs. Data Class: Understanding the Differences," Rocket. [Online]. Available: https://www.dhiwise.com/post/kotlin-class-vs-data-class-a-simple-comparison

[7] D. R.Musicant, "Kotlin in the Data Structures Classroom," pp. 1447–1448, 2026, doi: https://dl.acm.org/doi/proceedings/10.1145/ 3770761.3777203.